



AWS CodeBuild

Introduction

AWS CodeBuild is a **fully managed build service** that compiles source code, runs tests, and produces software packages ready for deployment.

- What is AWS CodeBuild?
- Why is it useful in CI/CD pipelines?
- How do you configure AWS CodeBuild in AWS Cloud?

This guide explains **how AWS CodeBuild automates the build process** in DevOps.

Section 1: Learn

What is AWS CodeBuild?

AWS CodeBuild is a **serverless** service that eliminates the need to manage and maintain build servers.

Why Use AWS CodeBuild?

Feature	Benefit
Fully Managed	No need to manage build infrastructure
Scalability	Scales automatically based on demand
Pay-as-You-Go	Charges only for the compute time used
Security	Supports AWS IAM roles for access control



Feature	Benefit
Integration	Works with GitHub, Bitbucket, AWS CodePipeline

AWS CodeBuild Workflow

Step	Purpose
Step 1: Source Code	Fetches code from AWS CodeCommit, GitHub, or S3
Step 2: Build Execution	Compiles and tests the code in a containerized environment
Step 3: Artifact Generation	Produces deployable output like .jar, .zip, or .docker images
Step 4: Deployment (Optional)	Can be integrated with AWS CodeDeploy or CodePipeline

Interesting Fact

AWS CodeBuild supports **custom build environments** using **Docker images**, allowing teams to define their own build configurations.

Section 2: Practice

1. Setting Up AWS CodeBuild

Step 1: Create a Build Project

1. Open **AWS Management Console** → Navigate to **AWS CodeBuild**.
2. Click **Create Build Project**.
3. Enter **Project Name** → Select **Source Code Provider** (AWS CodeCommit, GitHub, or S3).



4. Select **Environment** (Choose **Managed Image** or **Custom Docker Image**).

5. Set **Build Specification** to:

- Use an existing **buildspec.yml** file from the source code.

Step 2: Define a **buildspec.yml** File

Create a file **buildspec.yml** in your repository:

```
version: 0.2

phases:
  install:
    runtime-versions:
      java: corretto11
  build:
    commands:
      - mvn clean install
  post_build:
    commands:
      - echo "Build complete"
artifacts:
  files:
    - target/*.jar
```

Step 3: Start the Build

1. Click **Start Build** → Monitor the logs in **Build Output**.
2. If successful, the build artifact is stored in **Amazon S3**.



Why is this useful?

- Automates **code compilation, testing, and packaging**.
-

2. Integrating AWS CodeBuild with AWS CodePipeline

AWS CodeBuild can be part of a **CI/CD pipeline**.

Step 1: Create a Pipeline in AWS CodePipeline

1. Open **AWS CodePipeline** → Click **Create Pipeline**.
2. Select **Source: AWS CodeCommit or GitHub**.
3. Select **Build Stage: AWS CodeBuild**.
4. Choose **Deploy Stage: AWS CodeDeploy**.
5. Click **Create Pipeline**.

Now, every **code push** triggers an automatic build process.

Why is this important?

- Ensures **continuous integration and testing** before deployment.
-

3. Running Tests with AWS CodeBuild

AWS CodeBuild can **run unit and integration tests**.

Modify **buildspec.yml** to include testing:

```
version: 0.2
```

```
phases:
```

```
  install:
```

```
    runtime-versions:
```

```
      python: 3.8
```

```
  build:
```



commands:

- pytest tests/

post_build:

commands:

- echo "Tests completed"

Why is this useful?

- Ensures **code quality** before deployment.
-

Section 3: Know More

Frequently Asked Questions

1. **What programming languages does AWS CodeBuild support?**
 - Java, Python, Node.js, Go, Ruby, .NET, and more.
 2. **Can AWS CodeBuild deploy applications?**
 - No, but it integrates with **AWS CodeDeploy** and **CodePipeline** for deployment.
 3. **How do I monitor AWS CodeBuild logs?**
 - View logs in **AWS CloudWatch Logs** and **AWS Console**.
 4. **Can I use a custom build environment?**
 - Yes! AWS CodeBuild supports **custom Docker images**.
 5. **How is AWS CodeBuild billed?**
 - Based on **compute time used per minute**.
-



Conclusion

AWS CodeBuild **automates application builds and testing** without managing servers.

Start by **creating a CodeBuild project, defining buildspec files, and integrating it into CI/CD pipelines**, and soon you'll be **automating software builds efficiently in AWS!**